📚 Week 1: Introduction to Database Systems

บทนี้เน้นภาพรวมว่าฐานข้อมูลคืออะไร และมีสถาปัตยกรรมแบบไหนครับ

- **Database vs DBMS:** * **Database (DB):** แหล่งเก็บข้อมูลที่เป็นระเบียบ
    - **DBMS:** ซอฟต์แวร์ที่ใช้สร้าง จัดการ และเข้าถึงฐานข้อมูล (เช่น MySQL, Oracle)
- **ระดับของมุมมองข้อมูล (Data Abstraction Level):** ออกสอบบ่อย!
    1. **View level (สูงสุด):** ระดับที่ User หรือ Application มองเห็น (จับต้องได้ง่ายสุด)
    2. **Logical level (ตรงกลาง):** ระดับที่ DBMS มองเห็นว่ามีข้อมูลอะไรสัมพันธ์กันอย่างไร (พวก ERD ที่เราออกแบบ)
    3. **Physical level (ล่างสุด):** ระดับการเก็บข้อมูลลงฮาร์ดดิสก์จริงๆ (จับต้องไม่ได้)
- **ประเภทของผู้ใช้งาน (Types of Users):**
    - *Naïve users:* ผู้ใช้ทั่วไปที่ใช้ผ่านหน้าจอ (เช่น พนักงานแบงก์)
    - *Application programmers:* โปรแกรมเมอร์ที่เขียนโค้ดต่อกับ DB
    - *Sophisticated users:* ผู้ใช้เชี่ยวชาญที่เขียน SQL ดึงข้อมูลเอง (เช่น Data Analyst)
    - **Casual users:** ผู้ใช้ที่นานๆ ใช้ที แต่ต้องการรีพอร์ตแบบใหม่ๆ เสมอ (เช่น ผู้จัดการ) <- *ตัวนี้ชอบออกสอบ!*
- **คุณสมบัติ ACID ของ Transaction:** ท่องให้ขึ้นใจ!
    - **A - Atomicity:** ทำงานต้องเสร็จ 100% ถ้าไม่เสร็จต้องยกเลิกกลับเป็น 0% (ห้ามทำครึ่งๆ กลางๆ)
    - **C - Consistency:** ทำเสร็จแล้วข้อมูลต้องถูกต้องตามกฎเสมอ
    - **I - Isolation:** แต่ละ Transaction ทำงานแยกอิสระ ไม่กวนกันแม้จะรันพร้อมกัน
    - **D - Durability:** อัปเดตสำเร็จแล้ว ข้อมูลต้องอยู่ถาวรแม้ไฟดับ
- **DBMS Architecture:** มีแบบ 1-Tier (ทำทุกอย่างในเครื่องเดียว), 2-Tier (Client ต่อตรงหา DB Server), และ 3-Tier (มี Application Server มาคั่นกลาง)

🗺️ **Week 2: Entity Relationship Diagram (ERD)**

บทนี้คือหัวใจของการออกแบบโครงสร้างฐานข้อมูลครับ

- **ประเภทของ Entity (สี่เหลี่ยม):**
  - **Strong Entity (เส้นเดี่ยว):** อยู่ได้ด้วยตัวเอง มีคีย์หลัก (PK) ของตัวเอง
  - **Weak Entity (เส้นคู่):** อ่อนแอ อยู่ไม่ได้ถ้าไม่มี Entity อื่นให้พึ่งพา คีย์ของมันเรียกว่า "Partial Key"
- **ประเภทของ Attribute (วงรี):**
  - **Simple:** แบ่งย่อยไม่ได้แล้ว (เช่น เพศ)
  - **Composite:** แบ่งย่อยได้อีก (เช่น ชื่อ-นามสกุล แยกเป็น ชื่อต้น, นามสกุล)
  - **Single-valued:** มีค่าเดียว (เช่น รหัสนักศึกษา)
  - **Multi-valued (วงรีเส้นคู่):** มีได้หลายค่า (เช่น เบอร์โทรศัพท์) *ต้องระวังตอนแปลงเป็นตาราง!*
  - **Derived (วงรีเส้นประ):** ค่าที่ได้จากการคำนวณ (เช่น "อายุ" คำนวณมาจาก วันเกิด)
- **ข้อกำหนด (Constraints) และการมีส่วนร่วม (Participation):**
  - **Total Participation (เส้นคู่):** บังคับมีส่วนร่วม 100% (เช่น รถทุกคัน "ต้องมี" เจ้าของ)
  - **Partial Participation (เส้นเดี่ยว):** ไม่บังคับ (เช่น คนบางคนอาจจะไม่มีรถก็ได้)
- **ประเภทของ Keys:**
  - **Candidate Key:** แอตทริบิวต์ "ทุกตัว" ที่มีสิทธิ์จะเป็น Primary Key
  - **Primary Key (PK):** คีย์หลักที่ถูกเลือก (ห้ามซ้ำ, ห้ามเป็น NULL)
  - **Foreign Key (FK):** คีย์นอกที่ใช้เชื่อมโยงกับตารางอื่น
  - **Composite Key:** เอา PK มากกว่า 1 ตัวมามัดรวมกันเป็นคีย์เดียว

🗃️ Week 3: Relational Model (การแปลง ERD เป็นตาราง)

บทนี้คือการเอาภาพวาดมาทำเป็นตารางเพื่อเตรียมเขียนโค้ด

- **คำศัพท์:** Relation = Table (ตาราง), Column = Schema (โครงสร้าง), Row = Data (ข้อมูล/Tuple)
- **กฎการจัดการ Foreign Key (เวลาฝั่ง Parent ถูกลบ):**
  - **SET NULL:** ลบ Parent แล้วให้ข้อมูลฝั่ง Child กลายเป็นค่าว่าง
  - **CASCADE:** พ่อตายลูกตายตาม (ลบ Parent แล้วข้อมูล Child ที่เชื่อมอยู่โดนลบเกลี้ยง)
  - **RESTRICT:** ห้ามลบ Parent เด็ดขาดถ้ายังมี Child อ้างอิงอยู่
- **การแปลง ERD เป็นตาราง (Mapping):**
  - **1:M (One-to-Many):** เอา PK ของฝั่ง 1 ไปเป็น FK ในฝั่ง M เสมอ
  - **M:N (Many-to-Many):** ต้องแตกออกมาสร้าง "ตารางใหม่ตรงกลาง" โดยดึงเอา PK ของทั้งสองฝั่งมาเป็น FK (และมักจะรวมกันเป็น Composite Key)

🧬 Week 4-5: Enhanced ER Model (EER)

เมื่อระบบซับซ้อนขึ้น ต้องใช้แนวคิดแบบ OOP มาช่วยอธิบาย (เนื้อหานี้อยู่ในสไลด์ Week 5)

- **Sub Class และ Super Class:** นำไปสู่คอนเซปต์ของการ **สืบทอดคุณสมบัติ (Inheritance)**
  - *Super Class:* ตัวแม่ เช่น "ยานพาหนะ"
  - *Sub Class:* ตัวลูก เช่น "รถบรรทุก", "มอเตอร์ไซค์" (ลูกจะสืบทอดคุณสมบัติจากแม่มาทั้งหมด และมีคุณสมบัติเฉพาะตัวเพิ่มได้)
- **มุมมองการออกแบบ:**
  - **Specialization (Top-Down):** มองจากกลุ่มใหญ่แล้วแตกย่อยลงมา
  - **Generalization (Bottom-Up):** เห็นของย่อยๆ ที่คล้ายกัน เลยจับมัดรวมกันเป็นกลุ่มใหญ่ขึ้น
- **ระดับความสัมพันธ์ (Degree of Relationship):**
  - **Unary:** สัมพันธ์กับตัวเอง (เช่น พนักงานเป็นหัวหน้าของพนักงาน)
  - **Binary:** สัมพันธ์ 2 ทางปกติ (เช่น คน มี รถ)
  - **Ternary:** สัมพันธ์ 3 ทาง (เช่น พนักงาน โปรเจกต์ ทักษะ) *เหตุผลที่ต้องใช้ Ternary เพราะถ้าแยกเป็น Binary 3 คู่ เราจะไม่รู้เลยว่า "พนักงานคนนี้ ใช้ทักษะอะไร ในโปรเจกต์ไหนกันแน่"*